

Immersive Engagement: Building the “Wildtrace” Sonic World with AI Assistance

Haokun Yang¹ and Wanjun Yang¹

Sichuan Conservatory of Music, Chengdu, China
1663590016@qq.com, yangwanjun@163.com

Abstract. This paper addresses the high cost and long development cycles of custom audio plugins by proposing an agile workflow that couples Csound with large language models (LLMs). Building on this workflow, we developed *Wildtrace*, a quadraphonic spatial audio plugin suite driven by a family of generative algorithms. The core idea of the suite is to invert the usual premise of spatial audio design: rather than moving sound objects through space as an end in itself, it treats the spatial motion of sound as the engine that drives sonic effects such as Doppler shift, panning, granulation, and chorus, so that motion shapes the sound instead of merely positioning it. The paper systematically describes the development logic, the human–AI collaboration model, the layered technical architecture built around a replaceable global renderer, and the self-designed front-end interaction. We assess the approach against conventional C++ plugin development: although no systematic benchmarks were run, the qualitative evidence is encouraging—eight plugins were completed and tested in under one month at zero direct API cost, demonstrating a viable paradigm for agile audio prototyping under resource constraints.

Keywords: Csound, audio plugin, algorithmic control

1 Background and Objectives

In the absence of AI assistance, audio plugin development presents a very high barrier to entry. As large language models (LLMs) mature in their capacity to support development, it has become a trend for electronic musicians to build plugins with AI, yet AI-assisted development on conventional C++ frameworks suffers from bulky codebases, high token consumption, vulnerability to open-source license violations, difficulty in refactoring low-level DSP, and lengthy development cycles. This study therefore explores a workflow in which Csound and an LLM collaborate closely, and uses it to develop a spatial audio plugin suite named *Wildtrace*.

The central innovation of the suite lies in this: conventional spatial audio tools aim to move sound objects through space as an end in itself; building on this, the project lets the spatial motion of sound objects mimic the motion patterns of natural trajectories, and then drives the generation of effects such as Doppler, panning, granulation, and chorus. Motion is no longer the destination but the means of shaping sound, achieving—at a fraction of the cost of manual automation—artistic results that previously demanded the combination of many effect processors and extensive envelope drawing.

2 Inspiration and Design Aesthetics

The initial inspiration for the *Wildtrace* series came from the Boids flocking algorithm [4]. Having adopted it as a driver of the sound field, we added further algorithms to form a suite of effect plugins, aiming to create an embodied experience situated *inside* the system rather than at a spectator’s distance. In contrast to the notion of “seeking freedom within the sound world”, this research seeks to “build a world within the freedom of sound”. Under this principle, mix positioning is no longer a matter of controlling the coordinate parameters of a single object; instead, algorithms generate motion trajectories for sound objects, and these trajectories then participate in—and even drive—sound design, allowing effects such as Doppler, panning, granulation, and chorus to grow naturally out of motion. This core idea can be summarized as *immersive engagement*.

It resonates with Gottfried Michael Koenig’s notion of grammar-driven generation [7], extending the rule set into the spatial dimension. This orientation toward evoking the listener through motion also connects with Truax’s soundscape composition [13]: through recognizable sonic materials and

soundscapes, the listener forms an association with the work and becomes a co-creator of the sound world. On this basis, and with the aid of an LLM, we distilled a spatial audio plugin suite comprising seven algorithms plus one manual positioning tool.

3 Traditional Development Challenges and the AI-Collaborative Paradigm

The agile development workflow can be summarized in three stages: first, *base and interaction substrate*—guiding the LLM to generate a four-channel equal-power panning opcode (QuadRender) and dynamic Image controls, completing the base prototype STASIS; second, *stepwise iterative development*—using STASIS as a template and directing the LLM through prompts to extend the remaining plugins, avoiding overly long single-pass code; third, *human-AI collaboration and closed-loop correction*—when repeated AI revisions still produced errors, manual fixes were applied and passing code was fed back into the LLM context to drive subsequent iterations.

Under the conventional Cabbage workflow, musicians without a technical background are spared from confronting C++, yet must still cross a learning threshold; the intervention of LLMs let the author focus on product concept, signal-flow architecture, and front-end aesthetics. The study used Csound 6.18 as the audio engine [1,2] and Cabbage 2.9.0 as the development platform [3], relying throughout on web-based models: Gemini 3 Flash [8] generated the algorithms, Doubao Seedream 5.0 and Gemini Nano Banana 2 [10,9] generated the plugin backgrounds, and open-source fonts and knob textures completed the front-end UI; DeepSeek-V4 [11] and Doubao Seed 2.0 [10] were also used during coding.

Thanks to this workflow, the eight Wildtrace plugins were developed and tested in under one month at zero direct API cost (free tier).

4 Results and Limitations

4.1 Results

1. *Low-overhead agile paradigm*: the study validates an agile incubation path of “full-pipeline LLM assistance at low direct cost”—less boilerplate, faster iteration (a qualitative observation), while avoiding the open-source license and intellectual-property risks of AI-generated code. It is particularly friendly to developers without a C++ background who seek rapid prototyping.
2. *Extensible technical architecture*: the core renderer is implemented as a global Csound user-defined opcode (UDO) that can be reused wholesale or replaced, forming a layered architecture of front-end parameters → global renderer → instrument algorithms → level feedback. The stereo “depth render” version validated this architecture: the Y axis is reinterpreted as spatial depth, and dry/wet paths are handed to **reverb** for reverberation; with a multi-delay-driven Doppler algorithm embedded in the downmix chain, depth motion translates directly into pitch drift—with LFO modulation off, the depth components of Boids and Singularity still independently produce Doppler. Replacing the renderer is thus not merely swapping an opcode; the control front end must be rebuilt in step, delineating the boundary between reusable algorithm/effect logic and a replaceable sound-field layer.
3. *Self-designed front-end interaction*: to work around the offset of the native XY pad cursor centre, we built our own simulated pad and dynamic Image controls, displaying multiple sound sources on one screen and breaking with the dated interaction of native controls through fuller sound-field visualization.
4. *Motion-driven sonic design*: the eight Wildtrace plugins upgrade self-organizing motion from trajectory generators into engines of sonic generation—trajectories directly drive four-channel equal-power panning, and in the stereo chain they realize a natural chorus that replaces the LFO, a grain cloud, and a double Doppler, so that spatial motion itself participates in sound design, enacting the idea of immersive engagement. The suite is open-sourced under the MIT license at <https://github.com/nocalledname/Wildtrace> [14].

Table 1. Design overview of the Wildtrace plugin series.

No. Plugin	Quadraphonic Version	Stereo Version
1 STASIS	Manual positioning baseline: mouse, value-entry, and host-automation inputs; reference prototype with no algorithmic intervention	Reference prototype; the Y axis is reinterpreted as spatial depth, reference with the depth renderer control-prototype with no algorithmic reverb send, amplitude attenuation, high-frequency attenuation, and delay time
2 ORBITA	Continuous circle-to-ellipse or bit switching (Shape); the elliptical arc varies its angular velocity naturally	Same trajectory, producing a sense of depth variation
3 SINGULARITY	Bidirectional spiral trajectory + explicit doppler button; black-hole contraction and white-hole divergence shape the sense of distance	With the button off, spiral depth motion still produces slight Doppler; combined with the button it forms a double Doppler
4 ZEPHYR	LFO + data jitter driving back-and-forth traversal; fast traversal produces a grain stream	Same trajectory; depth wobble produces slight Doppler
5 LÉVY-CHASE	Lévy flight, power-law long-tailed step distribution	Same trajectory; the depth component of long-tail jumps and smooth gliding produces intermittent Doppler
6 LOOM	Row $\times$ column lattice weaving, serpentine traversal; the sound is scattered into N fixed-position grains	Algorithm as in the quadraphonic version; the traversal’s depth component is rendered through the depth renderer
7 SWARM-GRAIN	McCurdy delay-grain scheduling [12] + Gaussian spatial filtering, forming a centre-to-edge shaded grain cloud	Algorithm as in the quadraphonic version; the grain field’s depth distribution is rendered through the depth renderer
8 BOIDS	Boids flocking (separation / alignment / cohesion); followers track the leader	Flock depth motion drives the multi-delay chain: even with LFO modulation off, it keeps producing Doppler pitch drift, forming a natural chorus

Note: in the stereo “depth render” version the Y axis is reinterpreted as spatial depth and a multi-delay-driven Doppler algorithm is embedded in the downmix chain; the quadraphonic version performs equal-power spatial panning only.

4.2 Limitations and Future Work

The limitations are twofold: first, *limited compilation and distribution*—constrained by the free version of Cabbage, standalone plugin distribution and multi-host (DAW) compatibility are limited, and testing across a wider range of hardware is lacking; second, *graphics rendering coupled with threading*—the UI drawing thread and the back-end DSP run on the same thread and cannot be decoupled, while the complex self-organizing algorithms are computed dynamically in the top-level Csound logic rather than compiled in C++, raising CPU overhead and causing occasional interface stutter on low-spec hardware.

Future work has three directions: first, *multi-dimensional sound-field extension*—develop a binaural downmix based on the head-related transfer function (HRTF) and an Ambisonics version, and introduce a Z axis for three-dimensional mapping; second, *low-level compilation optimization*—compile custom opcodes in C++ and embed them in Csound to reduce dynamic computation overhead; third, *cross-platform distribution and ecosystem testing*—explore additional build channels such as Cabbage Pro and conduct broad compatibility testing across hosts (DAWs) and hardware configurations.

References

1. Yang, W.: Introduction to Csound Music Programming (in Chinese). Southwest China Normal University Press (2015)
2. Csound Canonical Reference Manual, <https://csound.com/docs/manual/index.html>
3. Cabbage Reference Manual, <https://cabbageaudio.com/docs/introduction/>
4. Reynolds, C.W.: Flocks, Herds, and Schools: A Distributed Behavioral Model. In: ACM SIGGRAPH Computer Graphics, vol. 21, no. 4, pp. 25–34 (1987). <https://www.red3d.com/cwr/boids/>
5. Tenney, J.: Meta+Hodos: A Phenomenology of 20th-Century Musical Materials and an Approach to the Study of Form, and META Meta+Hodos, 2nd edn. Frog Peak Music (1988). https://eamusic.dartmouth.edu/~larry/published_articles/tenney_monograph_soundings/19_META_HODOS.pdf
6. Roads, C.: The Computer Music Tutorial. MIT Press, Cambridge, MA (1996)
7. Koenig, G.M.: The Use of Computer Programmes in Creating Music. In: Music and Technology. UNESCO, Paris: La Revue Musicale (1970). https://koenigproject.nl/wp-content/uploads/2019/08/The_Use_of_Computer_Programmes_in_Creating_Music.pdf
8. Google DeepMind: Gemini 3 Flash [Model card] (2025). <https://deepmind.google/models/model-cards/gemini-3-flash>
9. Google DeepMind: Gemini 3.1 Flash Image (Nano Banana 2) [Model card] (2026). <https://deepmind.google/models/model-cards/gemini-3-1-flash-image>
10. ByteDance Seed: Doubao Seed 2.0 and Seedream 5.0 (large language model and image generation model) (2026). <https://seed.bytedance.com/>
11. DeepSeek: DeepSeek-V4 [Large language model] (2026). <https://www.deepseek.com/>
12. McCurdy, I.: Csound Realtime Examples. <http://iainmccurdy.org/csound.html>
13. Truax, B.: Soundscape, Acoustic Communication and Environmental Sound Composition. Contemporary Music Review 15(1), 49–65 (1996)
14. Wildtrace: An open-source Csound/Cabbage quadraphonic and stereo spatial audio plugin suite (MIT License). <https://github.com/nocalledname/Wildtrace>